

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 1



BÁO CÁO
HỆ ĐIỀU HÀNH NHÚNG

ĐỀ TÀI:
XÂY DỰNG HỆ THỐNG GIÁM SÁT VÀ CẢNH BÁO RÒ RỈ KHÍ GAS
TRONG PHÒNG NHÀ

Giảng viên: ThS. Lương Công Duẩn

Sinh viên thực hiện: Nguyễn Đức Cảnh – B22DCDT043

Đỗ Trọng Giáp – B22DCDT099

Tạ Ngọc Quý – B22DCDT255

Lê Anh Quốc – B22DCDT251

Lớp: D22DTMT02

Nhóm: 09

Hà Nội, 2026

MỤC LỤC

LỜI CẢM ƠN	3
CHƯƠNG 1: GIỚI THIỆU CHUNG	4
1.1. Mục tiêu của báo cáo	4
1.2. Lý do chọn đề tài.....	4
1.3. Tổng quan.....	4
1.3.1. Giới thiệu Buildroot- Hệ điều hành nhúng	4
1.3.2. Giới thiệu BeagleBone Black- Máy tính nhúng	5
1.4. Sơ lược về hệ thống.....	5
1.4.1. Sơ đồ khối của hệ thống	5
1.4.2. Bối cảnh nhu cầu thực tế.....	5
1.4.3. Giải pháp đề tài đề xuất	6
CHƯƠNG 2: THIẾT KẾ PHẦN CỨNG VÀ PHẦN MỀM	7
2.1. Các thiết bị và phần cứng sử dụng.....	7
2.1.1. BeagleBone Black	7
2.1.2. Mạch chuyển tín hiệu ADS1115	8
2.1.3. Cảm biến nhiệt độ, độ ẩm – DHT11	10
2.1.4. Cảm biến khí ga MQ-2.....	12
2.2. Giao tiếp và công nghệ phần mềm	13
2.2.1. Linux.....	13
2.2.2. Cấu hình Buildroot.....	13
2.2.3. I2C.....	17
2.2.4. Hướng dẫn Code Driver, Application	18
2.2.5. Autostart và đóng gói buildroot	23
CHƯƠNG 3: XÂY DỰNG HỆ THỐNG	24
3.1. Chức năng chính của hệ thống	24
3.2. Thử nghiệm thực tế hệ thống.....	25
CHƯƠNG 4: KẾT QUẢ, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN	26
4.1. Kết luận.....	26
4.2. Hướng phát triển	26
PHỤ LỤC: MÃ NGUỒN & GITHUB	27
TÀI LIỆU THAM KHẢO.....	28

LỜI CẢM ƠN

Nhóm thực hiện xin chân thành cảm ơn thầy hướng dẫn đã tận tình chỉ bảo, định hướng và hỗ trợ trong suốt quá trình nghiên cứu và hoàn thành đề tài. Những kiến thức, kỹ năng và kinh nghiệm mà nhóm thu nhận được từ quá trình thực hiện đề tài là vô cùng quý giá.

Nhóm cũng xin gửi lời cảm ơn đến bộ môn và nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, thiết bị và tài liệu phục vụ cho quá trình học tập và nghiên cứu.

Cuối cùng, xin cảm ơn các bạn trong nhóm đã cùng nhau làm việc nghiêm túc, đoàn kết và hỗ trợ lẫn nhau để hoàn thành tốt đề tài.

BẢNG PHÂN CÔNG NGHIÊM VỤ

Phân công nhiệm vụ	Họ tên
Nguyễn Đức Cảnh (C)	Viết app điều khiển driver, thuyết trình, làm word
Đỗ Trọng Giáp	Viết driver đọc cảm biến MQ2, LCD 20x04
Lê Anh Quốc	Viết driver đọc cảm biến DHT11, làm ppt
Tạ Ngọc Quý	Viết driver buzzer, làm ppt

CHƯƠNG 1: GIỚI THIỆU CHUNG

1.1. Mục tiêu của báo cáo

Đề tài hướng tới việc xây dựng hệ thống giám sát và cảnh báo rò rỉ khí gas sử dụng máy tính nhúng BeagleBone Black. Hệ thống thực hiện đọc và xử lý dữ liệu từ cảm biến MQ2 nhằm phát hiện nồng độ khí gas vượt ngưỡng cho phép. Bên cạnh đó, cảm biến DHT11 được tích hợp để thu thập thêm thông tin nhiệt độ và độ ẩm môi trường, hỗ trợ quá trình giám sát. Khi phát hiện nồng độ khí gas nguy hiểm, hệ thống sẽ kích hoạt còi báo động để cảnh báo an toàn. Thông qua đề tài, nhóm có thể rèn luyện kỹ năng lập trình nhúng, giao tiếp phần cứng, xây dựng driver và tích hợp hệ thống vào một bài toán thực tế.

1.2. Lý do chọn đề tài

Trong đời sống hiện nay, các sự cố rò rỉ khí gas trong gia đình, phòng thí nghiệm hay khu vực công nghiệp có thể gây ra cháy nổ và ảnh hưởng nghiêm trọng đến an toàn con người cũng như tài sản. Vì vậy, việc xây dựng một hệ thống có khả năng phát hiện sớm và cảnh báo kịp thời khi nồng độ khí gas vượt ngưỡng là rất cần thiết. Bên cạnh đó, sự phát triển mạnh mẽ của các hệ thống nhúng và hệ điều hành Linux trên nền tảng phần cứng nhỏ gọn đã mở ra nhiều ứng dụng thực tế trong lĩnh vực giám sát và tự động hóa.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài xây dựng hệ thống giám sát và cảnh báo rò rỉ khí gas sử dụng BeagleBone Black kết hợp với Buildroot nhằm nghiên cứu và áp dụng các kiến thức về lập trình nhúng, Linux Embedded, driver kernel và giao tiếp phần cứng vào một bài toán thực tế. Đề tài không chỉ giúp nâng cao khả năng thiết kế hệ thống nhúng hoàn chỉnh từ mức kernel đến user space mà còn tạo tiền đề cho việc phát triển các hệ thống giám sát thông minh trong tương lai.

1.3. Tổng quan

1.3.1. Giới thiệu Buildroot- Hệ điều hành nhúng

Buildroot là một công cụ mã nguồn mở dùng để tạo hệ thống Linux nhúng tối giản, chuyên biệt cho phần cứng cụ thể. Buildroot hỗ trợ biên dịch và đóng gói kernel, bootloader, thư viện hệ thống, cũng như các ứng dụng người dùng thành một bộ firmware hoàn chỉnh có thể chạy trên các thiết bị nhúng.

Với khả năng tùy biến cao, Buildroot cho phép người phát triển:

- Chọn lựa và cấu hình thành phần hệ thống phù hợp với phần cứng mục tiêu.
- Tạo root filesystem gọn nhẹ, chỉ bao gồm các thành phần cần thiết.

- Dễ dàng tích hợp ứng dụng người dùng, module nhân (kernel module), driver thiết bị tự viết.
- Hỗ trợ nhiều kiến trúc vi xử lý như ARM, x86, MIPS,...

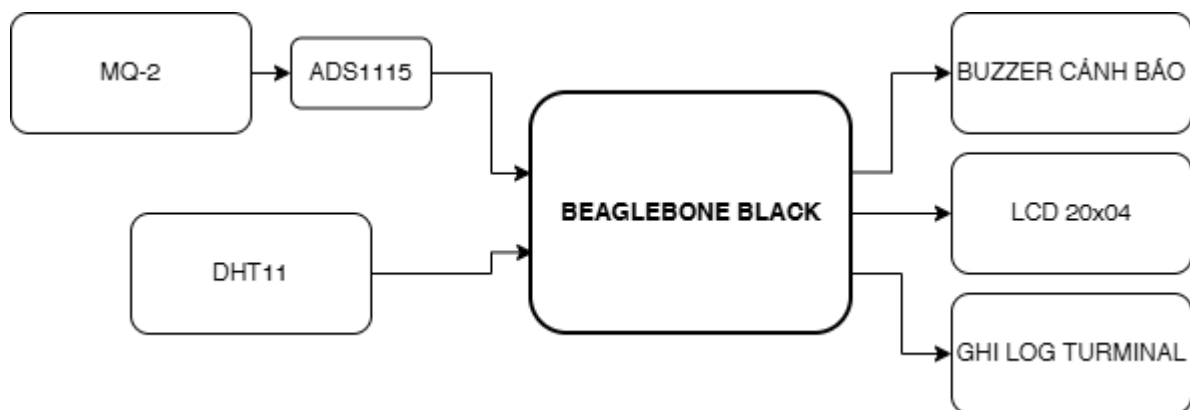
1.3.2. Giới thiệu BeagleBone Black- Máy tính nhúng

BeagleBone Black (BBB) là một máy tính nhúng đơn bo mạnh mẽ dựa trên vi xử lý ARM Cortex-A8. Với khả năng chạy hệ điều hành Linux và tích hợp sẵn các giao tiếp như GPIO, I2C, UART, SPI,... BBB là nền tảng lý tưởng cho các ứng dụng nhúng thời gian thực.

1.4. Sơ lược về hệ thống

1.4.1. Sơ đồ khối của hệ thống

Sơ đồ:



Giải thích:

- Sau khi start BeagleBone Black, boot thành công vào kernel thì Application sẽ tự động chạy và tự động insmod 4 driver bao gồm MQ2, Buzzer, DHT11 và LCD
- BeagleBone Black nhận thông tin từ driver và in ra màn hình monitor, đồng thời trong quá trình đó thì cũng sẽ tự tạo ra 1 LOG để theo dõi quá trình chạy của Application sau này.
- Trong quá trình chạy, driver buzzer và driver MQ2 phụ thuộc vào nhau, nếu nồng độ khí của MQ2 driver vượt ngưỡng thì buzzer driver sẽ bật và tắt nếu nồng độ khí của MQ2 trở về mức an toàn đồng thời hiển thị giá trị cảm biến đọc được lên màn hình LCD.

1.4.2. Bối cảnh nhu cầu thực tế

Trong môi trường nhà ở hiện nay, nguy cơ rò rỉ khí gas là một vấn đề nghiêm

trọng, có thể gây ra cháy nổ, ảnh hưởng đến an toàn con người và tài sản. Việc phát hiện và xử lý kịp thời các hiện tượng bất thường về khí gas là điều hết sức cần thiết.

1.4.3. Giải pháp đề tài đề xuất

Đề tài xây dựng một hệ thống giám sát khí gas hoạt động hoàn toàn độc lập trên nền tảng BeagleBone Black, bao gồm:

- Cảm biến MQ2: Đo nồng độ khí gas (LPG, butan, propane...).
- Cảm biến DHT11: Đo nhiệt độ và độ ẩm môi trường.
- Buzzer: Thiết bị cảnh báo khi phát hiện khí gas vượt ngưỡng.
- LCD 20x04: Hiển thị giá trị của cảm biến và cảnh báo vượt ngưỡng
- BBB: Thu thập, xử lý dữ liệu và điều khiển thiết bị.
- Ứng dụng nhúng: Được lập trình bằng ngôn ngữ C, chạy nền trên hệ điều hành do Buildroot tạo ra.

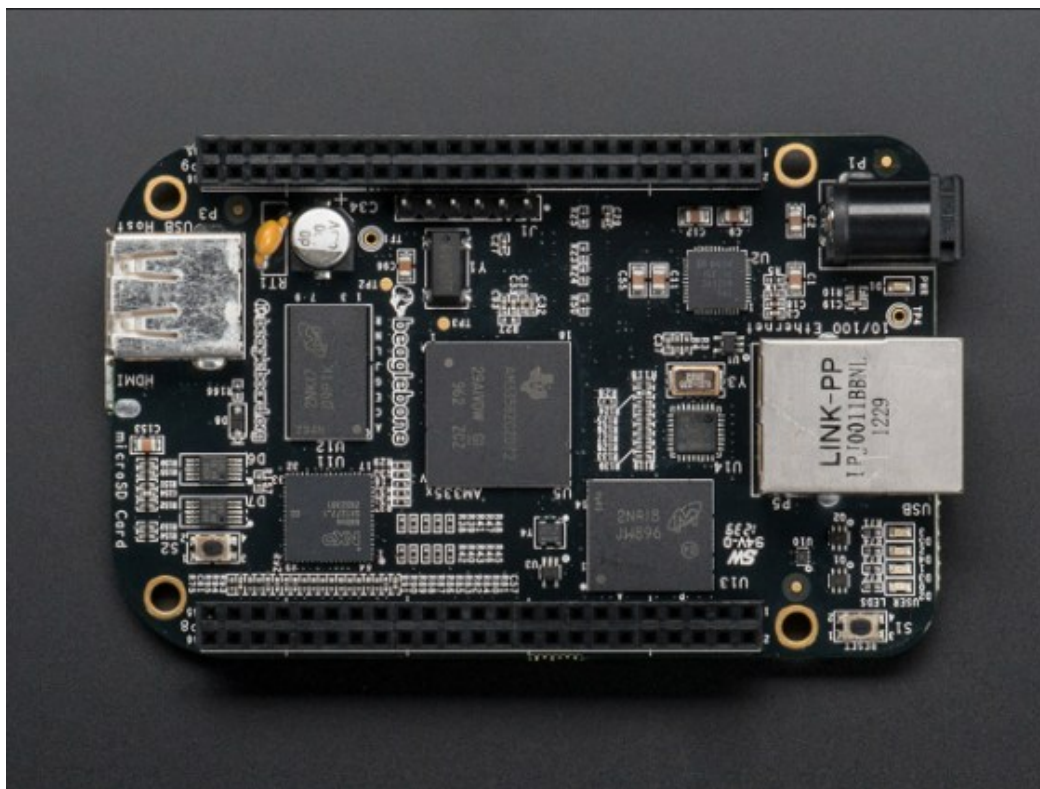
Kết luận chương 1

Chương này đã giới thiệu khái quát các công nghệ nền tảng được sử dụng trong đề tài, bao gồm Buildroot- công cụ xây dựng hệ điều hành nhúng, BeagleBone Black- nền tảng phần cứng xử lý trung tâm, và tổng quan hệ thống giám sát khí gas. Đây là cơ sở để phát triển một hệ thống giám sát môi trường đơn giản, hiệu quả và có tính ứng dụng thực tiễn cao.

CHƯƠNG 2: THIẾT KẾ PHẦN CỨNG VÀ PHẦN MỀM

2.1. Các thiết bị và phần cứng sử dụng

2.1.1. BeagleBone Black



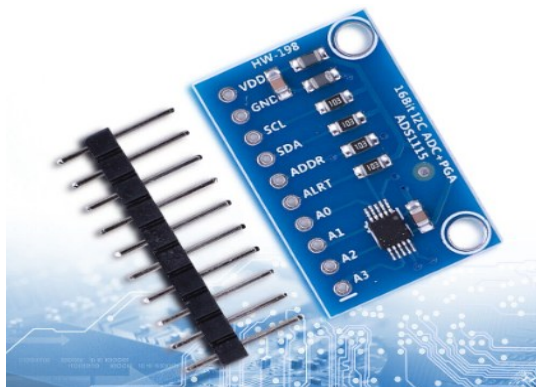
Hình 2.1. BeagleBone Black

Thông số và tính năng:

- Vi xử lý: Texas Instruments Sitara AM335x ARM Cortex-A8, tốc độ 1GHz, tích hợp đơn nhân, hỗ trợ NEON và đồ họa 3D (GPU PowerVR SGX530)
- RAM: 512MB DDR3L
- Bộ nhớ trong: 4GB eMMC Flash, hỗ trợ mở rộng qua thẻ MicroSD
- Cổng USB:
 - 1 x USB 2.0 Host (A)
 - 1 x USB 2.0 Client (Mini-B) dùng để cấp nguồn và giao tiếp
- Kết nối mạng: Cổng Ethernet 10/100 Mbps
- GPIO mở rộng: 2 đầu header 46 chân, tổng cộng 65 chân GPIO lập trình được
- Giao tiếp ngoại vi:
 - UART: 5 kênh
 - I2C: 2 kênh
 - SPI: 2 kênh

- CAN: 2 kênh
- PWM: 7 kênh
- ADC: 7 kênh 12-bit
- Video/Audio Out:
 - HDMI micro (hỗ trợ âm thanh và video)
 - Audio analog qua cape mở rộng
- Lưu trữ: eMMC 4GB và thẻ MicroSD (khởi động được từ SD)
- Debug & Giao tiếp lập trình:
 - Cổng UART debug (UART0) qua header
 - Hỗ trợ giao tiếp JTAG
- Hệ điều hành hỗ trợ: Debian GNU/Linux (mặc định), Buildroot, Ubuntu, Yocto Linux
- Tính năng bổ sung:
 - RTC (Real-Time Clock) tích hợp
 - Hỗ trợ boot từ USB, eMMC hoặc microSD
- LED trạng thái lập trình được
- Kết nối mở rộng: Hỗ trợ hơn 20 loại cape mở rộng cho LCD, motor, CAN, relay,...
- Yêu cầu nguồn điện:
 - 5V/2A qua jack DC (barrel jack)
 - Hoặc 5V qua cổng mini-USB (dòng hạn chế)

2.1.2. Mạch chuyển tín hiệu ADS1115



Hình 2.2. ADS1115

Module ADS1115 là bộ chuyển đổi tương tự – số (ADC – Analog to Digital Converter) độ phân giải cao, giao tiếp qua I2C, thường được sử dụng trong các hệ thống

nhúng để đọc tín hiệu analog từ cảm biến với độ chính xác vượt trội. Trong đề tài này, ADS1115 đóng vai trò chuyển đổi tín hiệu tương tự từ cảm biến khí gas (như MQ2) sang tín hiệu số để ESP32 hoặc BeagleBone Black có thể xử lý. Với độ phân giải 16-bit và nhiều kênh đầu vào, ADS1115 giúp đảm bảo việc đo lường nồng độ khí gas được chính xác, ổn định và dễ tích hợp trong môi trường SCADA.

Thông số và tính năng:

- Điện áp hoạt động: 2.0V – 5.5V DC
- Dòng tiêu thụ:
 - ~150 μ A khi hoạt động
 - < 1 μ A ở chế độ ngủ
- Giao tiếp: I2C (tốc độ đến 860 mẫu/giây)
- Độ phân giải: 16-bit (giá trị từ -32768 đến 32767)
- Số kênh analog: 4 kênh đơn (A0 – A3) hoặc 2 kênh vi sai
- Gain (khuếch đại): Có thể chọn từ $\pm 0.256V$ đến $\pm 6.144V$
- Tốc độ mẫu (SPS): 8 đến 860 samples/second
- Địa chỉ I2C: 0x48 (có thể thay đổi bằng cách nối chân ADDR)
- Điện áp tham chiếu nội: Có sẵn, không cần thêm ngoài
- Bộ khuếch đại tín hiệu nội (PGA): Giúp tăng độ nhạy cho cảm biến yếu
- Kích thước module: Khoảng 25mm x 15mm

Ứng dụng:

- Đọc tín hiệu cảm biến analog như MQ2, pH, ánh sáng, áp suất,...
- Giám sát điện áp pin, dòng điện, điện trở biến thiên
- Ứng dụng trong thiết bị y tế, IoT, SCADA công nghiệp
- Lý tưởng cho các hệ thống cần đo analog chính xác, chống nhiễu

Sơ đồ đấu nối:

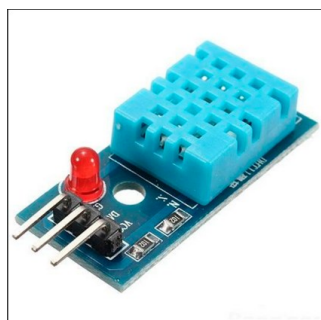
- VCC: Nối nguồn 3.3V hoặc 5V từ ESP32, BBB,...
- GND: Nối đất chung với hệ thống
- SCL: Chân clock I2C, kết nối với GPIO I2C SCL của ESP32 hoặc BBB
- SDA: Chân dữ liệu I2C, kết nối với GPIO I2C SDA
- ADDR (tùy chọn): Dùng để thay đổi địa chỉ I2C (kết nối GND, VCC hoặc treo)

- A0 – A3: Đầu vào analog (tín hiệu cảm biến analog sẽ nối vào đây)

Nguyên lý hoạt động:

- ADS1115 nhận tín hiệu điện áp analog từ cảm biến (ví dụ: MQ2) qua các chân A0–A3.
- Bộ chuyển đổi tích hợp sẽ khuếch đại tín hiệu (nếu cần) thông qua PGA và thực hiện chuyển đổi sang số với độ phân giải 16-bit.
- Việc điều khiển module (chọn kênh, cấu hình gain, tốc độ lấy mẫu...) được thực hiện thông qua giao tiếp I2C giữa vi điều khiển và ADS1115.
- Mỗi lần đọc dữ liệu, vi điều khiển gửi lệnh qua I2C để yêu cầu đo trên một kênh cụ thể, chờ thời gian chuyển đổi (~1–2ms tùy cấu hình), rồi đọc 2 byte dữ liệu trả về từ module.
- Giá trị số này sẽ được xử lý để tính toán điện áp đầu vào, và từ đó quy đổi thành nồng độ khí gas (trong trường hợp MQ2) hoặc các đại lượng tương ứng với cảm biến khác.

2.1.3. Cảm biến nhiệt độ, độ ẩm – DHT11



Hình 2.3. DHT11 sensor

Cảm biến nhiệt độ và độ ẩm DHT11 là một phần tử cảm biến môi trường kỹ thuật số, được sử dụng rộng rãi trong các hệ thống giám sát và điều khiển tự động. Thiết bị có khả năng đo và truyền dữ liệu nhiệt độ – độ ẩm với độ chính xác vừa phải, phù hợp cho các ứng dụng trong nhà hoặc hệ thống cảnh báo môi trường cơ bản. Trong đề tài này, DHT11 được tích hợp vào hệ thống sử dụng ESP32 hoặc BeagleBone Black, cho phép thu thập thông tin môi trường nhằm đánh giá điều kiện không khí, cảnh báo hoặc làm đầu vào cho các thuật toán điều khiển (ví dụ như kích hoạt quạt làm mát)

Thông số và tính năng:

- Điện áp hoạt động: 3.3V – 5.5V DC
- Dòng tiêu thụ:

- ~0.5 mA khi đo
- ~50 μ A khi chờ
- Dải đo nhiệt độ: 0°C – 50°C
- Độ chính xác nhiệt độ: $\pm 2^\circ\text{C}$
- Dải đo độ ẩm: 20% – 90% RH (độ ẩm tương đối)
- Độ chính xác độ ẩm: $\pm 5\%$ RH
- Chu kỳ lấy mẫu: Tối thiểu 1 giây (1 Hz)
- Loại kết nối: Digital 1-wire (chỉ 1 chân dữ liệu)
- Kích thước cảm biến: 15.5mm x 12mm x 5.5mm
- Cấu tạo: Tích hợp cảm biến nhiệt độ (NTC) và cảm biến độ ẩm điện trở
- Vật liệu vỏ: Nhựa ABS đúc kín, có khe thoát khí
- Tín hiệu ra: Gói 40-bit số (8+8 bit độ ẩm, 8+8 bit nhiệt độ, 8 bit checksum)
- Tốc độ phản hồi: ~2 giây cho một chu kỳ đo hoàn chỉnh
- Tuổi thọ: Trên 10.000 chu kỳ đo ổn định
- Khả năng ứng dụng:
 - Giám sát nhiệt độ và độ ẩm không khí
 - Làm đầu vào cho hệ thống kiểm soát khí hậu (climate control)
 - Tích hợp trong các thiết bị IoT, nhà thông minh, SCADA
 - Phối hợp với cảm biến khí gas để đánh giá môi trường nguy hiểm
 - Kích hoạt cảnh báo hoặc các thiết bị phản ứng như quạt hút khí, còi báo, ...

Sơ đồ đấu nối:

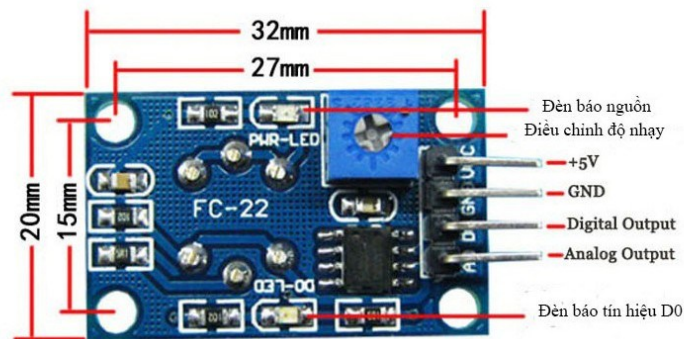
- Chân 1 – VCC: Nối nguồn 3.3V hoặc 5V từ vi điều khiển (ESP32, Arduino, BBB...)
- Chân 2 – DATA: Nối tới GPIO bất kỳ, nên dùng thêm điện trở kéo lên (pull-up resistor) 10k Ω giữa DATA và VCC
- Chân 3 – NC: Không kết nối (Not Connected)
- Chân 4 – GND: Nối GND của vi điều khiển

Nguyên lý hoạt động:

- Khi cấp nguồn, cảm biến DHT11 khởi động trong khoảng 1 giây.
- Vi điều khiển sẽ gửi một xung yêu cầu (start signal) trên chân DATA kéo xuống mức thấp trong ít nhất 18ms.

- Cảm biến phản hồi bằng một xung tín hiệu xác nhận, sau đó truyền dữ liệu số 40 bit, bao gồm:
 - 16-bit độ ẩm (nguyên + thập phân)
 - 16-bit nhiệt độ (nguyên + thập phân)
 - 8-bit kiểm tra lỗi (checksum)
- Mỗi bit được mã hóa theo độ rộng xung (pulse width modulation), vì điều khiển cần đo thời gian chính xác để giải mã đúng.
- Dữ liệu sau khi nhận sẽ được xử lý để hiển thị hoặc sử dụng làm điều kiện điều khiển hệ thống.

2.1.4. Cảm biến khí ga MQ-2



Hình 2.4. Cảm biến khí ga MQ-2

Cảm biến khí gas MQ-2 sử dụng phân tử SnO_2 có độ dẫn điện thấp hơn trong không khí sạch, khi khí dễ cháy tồn tại, cảm biến có độ dẫn điện cao hơn, nồng độ chất dễ cháy càng cao thì độ dẫn điện của SnO_2 sẽ càng cao và được tương ứng chuyển đổi thành mức tín hiệu điện.

Cảm biến khí gas MQ-2 là cảm biến khí có độ nhạy cao với LPG, Propane và Hydrogen, mê-tan (CH_4) và hơi dễ bắt lửa khác, với chi phí thấp và phù hợp cho các ứng dụng khác nhau.

Thông số và tính năng:

- Nguồn hoạt động: 5V
- Loại dữ liệu: Analog
- Phạm vi phát hiện rộng
- Tốc độ phản hồi nhanh và độ nhạy cao
- Mạch đơn giản
- Ổn định khi sử dụng trong thời gian dài

Cảm biến xuất ra cả hai dạng tín hiệu là Analog và Digital, tín hiệu Digital có thể điều chỉnh mức báo bằng biến trở.

2.2. Giao tiếp và công nghệ phần mềm

2.2.1. Linux

Linux là một hệ điều hành mã nguồn mở, miễn phí, được phát triển dựa trên kiến trúc Unix. Được khởi xướng bởi **Linus Torvalds** vào năm 1991, Linux hiện nay đã trở thành một trong những nền tảng phổ biến nhất trong các hệ thống máy chủ, thiết bị nhúng, hệ thống nhúng công nghiệp, điện thoại thông minh (như Android), và cả các siêu máy tính.

Linux có nhiều ưu điểm nổi bật như tính ổn định cao, khả năng bảo mật tốt, khả năng tùy biến linh hoạt và cộng đồng phát triển rộng lớn. Với nhân (kernel) mạnh mẽ và hệ sinh thái phong phú, Linux là lựa chọn hàng đầu cho các kỹ sư phần mềm, quản trị viên hệ thống và nhà phát triển trên toàn thế giới.

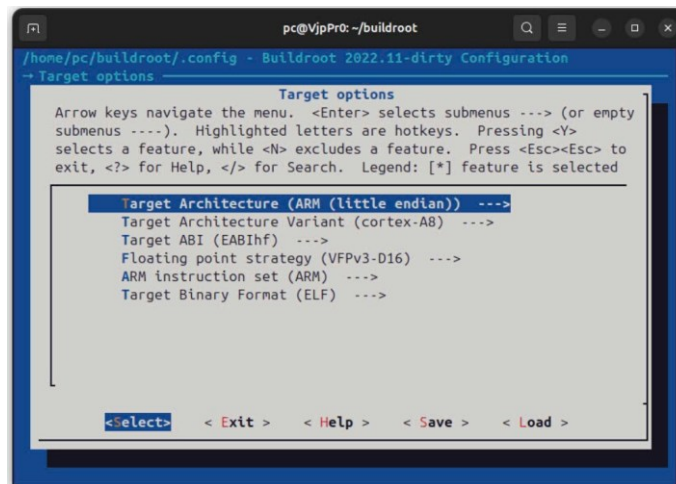
Một điểm đáng chú ý của Linux là sự hiện diện của các bản phân phối (distribution) như Ubuntu, Debian, CentOS, Fedora, Arch Linux,... giúp người dùng có thể lựa chọn hệ thống phù hợp với nhu cầu cụ thể của mình, từ người mới bắt đầu đến các chuyên gia hệ thống.

2.2.2. Cấu hình Buildroot

Buildroot là một công cụ mạnh mẽ để tạo ra hệ điều hành Linux nhúng tùy chỉnh. Trong quá trình cấu hình (make menuconfig), người dùng có thể điều chỉnh chi tiết từng thành phần của hệ thống, bao gồm kiến trúc phần cứng, toolchain, kernel, bootloader, hệ thống tập tin gốc và các tiện ích hệ thống.

Target Options:

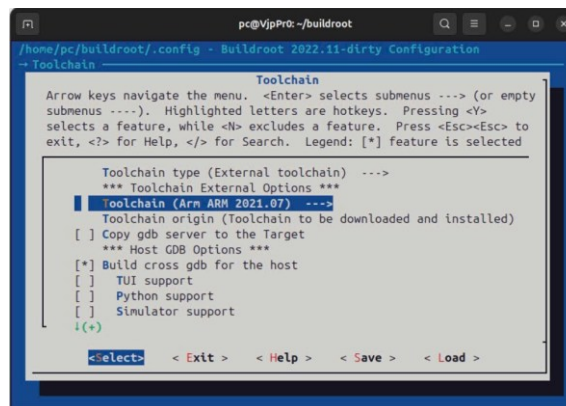
- Target Architecture: ARM(little endian)
- Target Architecture Variant: cortex-A8
- Target ABI: EABIhf
- Floating point strategy: VFPv3-D16
- ARM instruction set: ARM
- Target Binary Format: ELF



Hình 2.5. Target options

Toolchain:

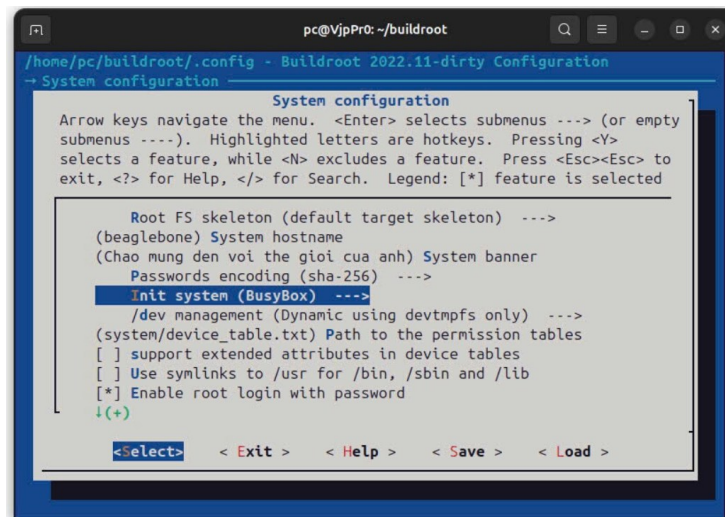
- Toolchain type: External toolchain
- Toolchain: phiên bản Arm ARM 2021.07



Hình 2.6. Toolchain

System configuration:

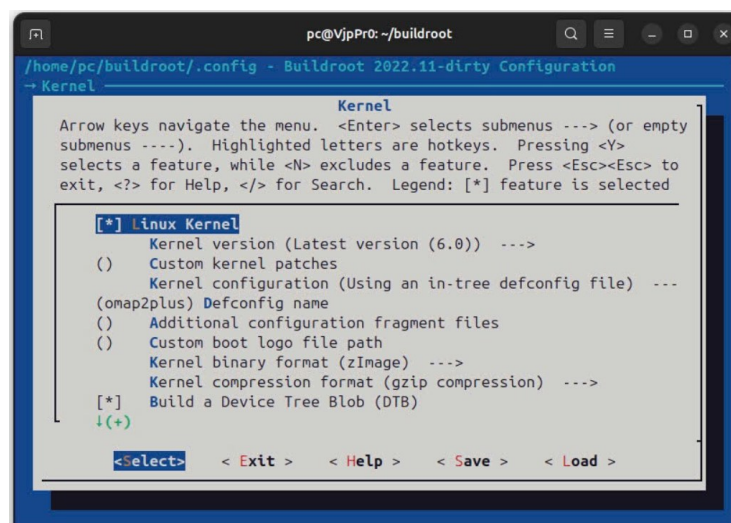
- System name: đặt tên tùy biến
- System banner: đặt tên tùy biến
- Init system: chọn BusyBox
- Enable root login with password: có thể chọn nếu muốn
- Path to permission tables: system/device_table.txt



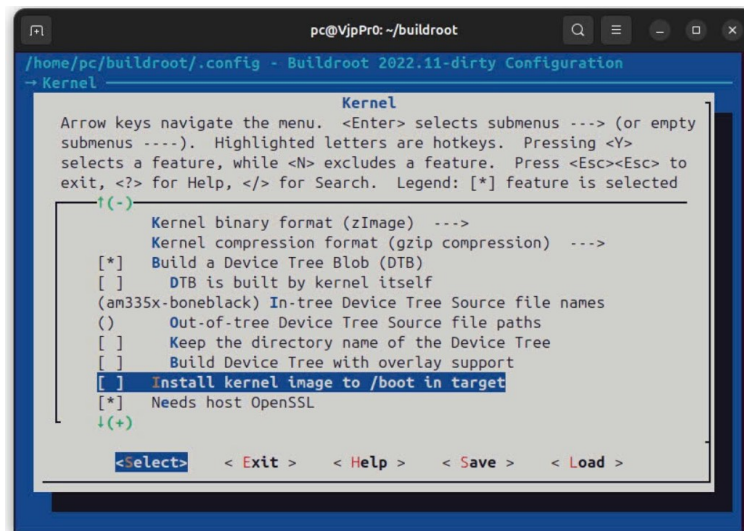
Hình 2.7. System configuration

Kernel

- Kernel version: Lastest version (6.0)
- Defconfig name: omap2plus
- Kernel binary format: zImage
- Build a Device Tree Blob(DTB): In-tree Device Tree Source file names (am335x-boneblack)



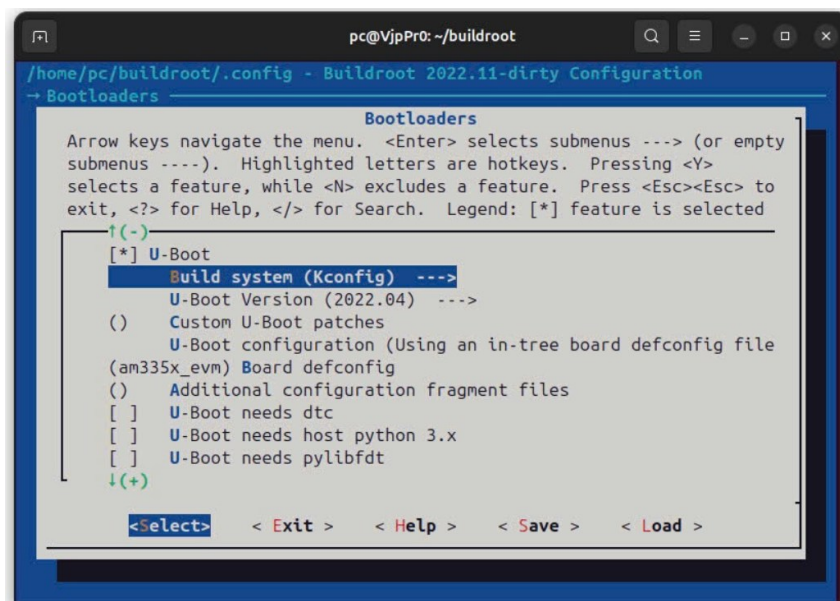
Hình 2.8.a. Kernel



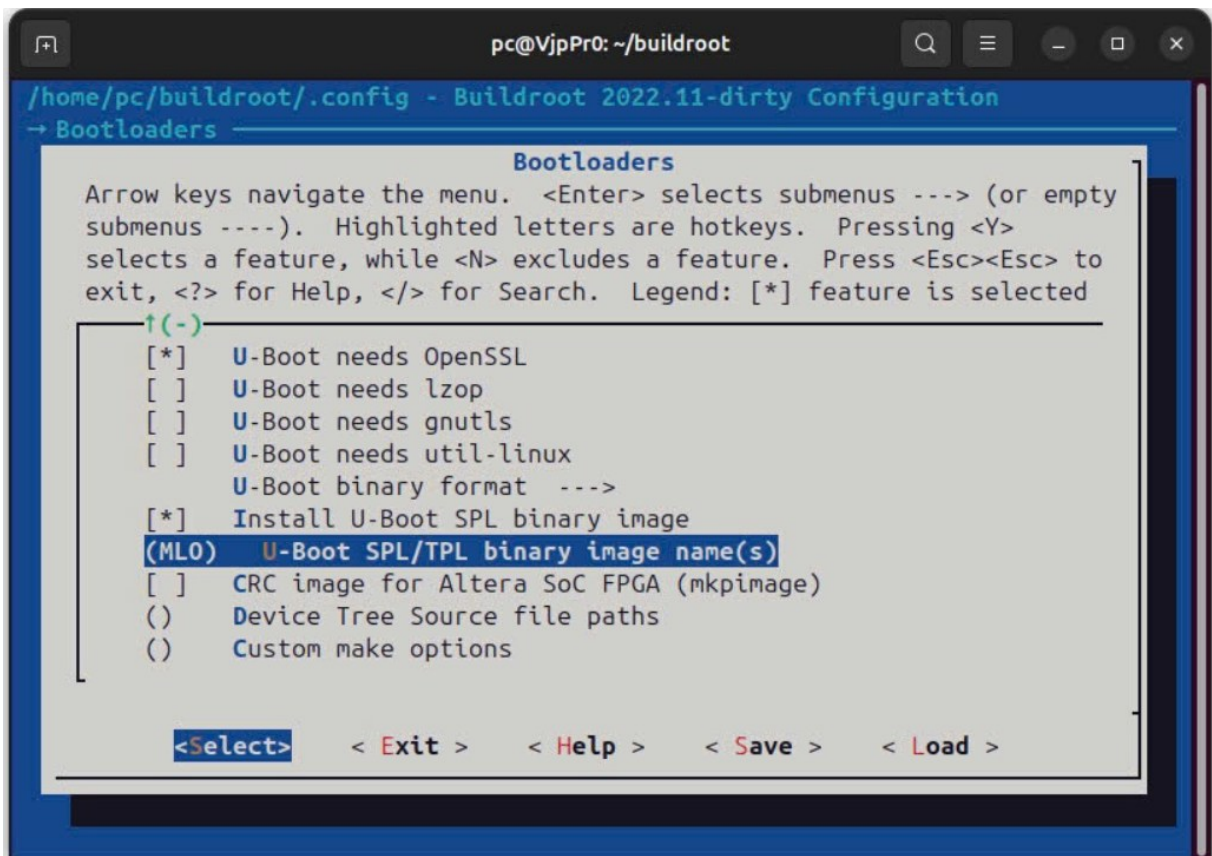
Hình 2.8.b. Kernel

Bootloader

- Build system: Kconfig
- U-boot Version: 2022.04
- U-boot configuration: using an in-tree board defconfig file
- Board defconfig: am335x_evm
- U-boot SPL/TPL binary image name(s): MLO

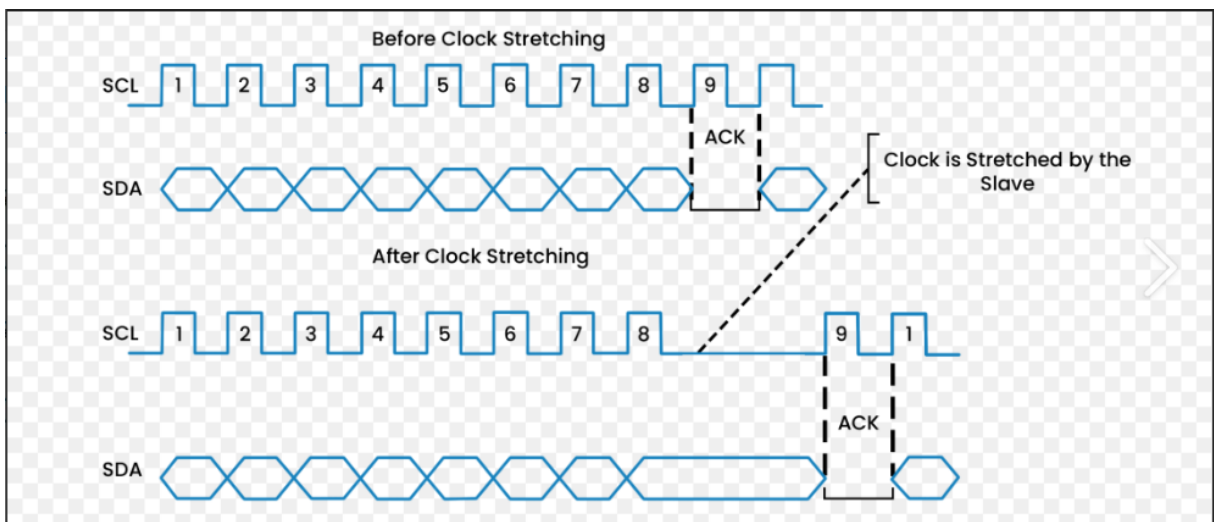


Hình 2.9.a. Bootloader



Hình 2.9.b. Bootloader

2.2.3. I2C



Hình 2.10. I2C

Hệ thống sử dụng giao thức I2C (Inter-Integrated Circuit) để truyền dữ liệu giữa BeagleBone Black (BBB) và các module cảm biến ngoại vi. I2C là một giao tiếp nối tiếp đồng bộ hai dây (SCL – Serial Clock và SDA – Serial Data), cho phép kết nối nhiều thiết bị với chỉ hai dây tín hiệu, giúp tối ưu hóa thiết kế mạch và giảm số lượng chân cần sử dụng trên vi điều khiển hoặc máy tính nhúng như BBB. Ứng dụng trong đề tài:

Ứng dụng trong đề tài:

- BeagleBone Black (BBB) đóng vai trò là master I2C, thực hiện giao tiếp với module ADC ADS1115 để đọc dữ liệu từ cảm biến khí gas MQ2.
- MQ2 là cảm biến khí gas phát ra tín hiệu analog tỷ lệ với nồng độ khí (butan, propane, methane, khói...). Tuy nhiên, BBB không có bộ ADC tích hợp sẵn với độ phân giải cao nên cần sử dụng ADS1115 để chuyển đổi tín hiệu analog sang số.
- ADS1115 là bộ chuyển đổi analog-số 16-bit có giao tiếp I2C, giúp thu thập tín hiệu từ MQ2 với độ chính xác cao. Tín hiệu sau khi được chuyển đổi sẽ được BBB thu nhận để xử lý, so sánh với ngưỡng an toàn, và quyết định kích hoạt còi hoặc điều khiển quạt hút khí.
- Việc truyền nhận dữ liệu giữa BBB và ADS1115 qua I2C được lập trình thông qua driver I2C trong nhân Linux hoặc sử dụng thư viện giao tiếp ở tầng ứng dụng (userspace).
- Giao tiếp I2C còn cho phép mở rộng hệ thống: BBB có thể kết nối đồng thời nhiều thiết bị như cảm biến nhiệt độ/độ ẩm, LCD, bộ mở rộng cổng I/O,... chỉ qua cùng một bus, giúp hệ thống SCADA linh hoạt, gọn nhẹ và dễ mở rộng.
- Quá trình truyền dữ liệu I2C được theo dõi, kiểm tra bằng các log hệ thống hoặc công cụ phân tích tín hiệu (logic analyzer), phục vụ quá trình debug và tối ưu hóa vận hành.

2.2.4. Hướng dẫn Code Driver, Application

Driver DHT11 (Đọc nhiệt độ và độ ẩm)

Driver DHT11 được xây dựng với mục đích giao tiếp trực tiếp với cảm biến DHT11 từ kernel nhằm thu thập dữ liệu nhiệt độ và độ ẩm của môi trường xung quanh. Hệ thống được triển khai trên nền tảng BeagleBone Black và sử dụng phương pháp ánh xạ thanh ghi GPIO thông qua hàm `ioremap` để truy xuất và điều khiển chân GPIO ở mức thấp.

Cảm biến DHT11 hoạt động dựa trên giao thức 1-wire phi tiêu chuẩn, vì vậy driver phải tự xử lý quá trình gửi tín hiệu bắt đầu và đọc dữ liệu bit theo thời gian thực. Trong cấu trúc driver, một node thiết bị ký tự `/dev/dht11` được tạo ra để user space có thể giao tiếp với kernel module. Driver cài đặt hàm `write` để nhận các lệnh điều khiển từ người dùng dưới dạng chuỗi "ON" hoặc "OFF". Khi nhận "ON", driver sẽ khởi động quá trình đọc dữ liệu từ cảm biến DHT11 và lưu kết quả vào buffer nội bộ; ngược lại, khi nhận "OFF", quá trình đọc cảm biến sẽ bị dừng lại. Ngoài ra, hàm `read` được sử dụng để trả về giá trị nhiệt độ và độ ẩm hiện tại cho user space dưới dạng chuỗi có định dạng `T:<giá trị nhiệt độ> H:<giá trị độ ẩm>`

Ioremap (ánh xạ gpio):

```

#define GPIO1_BASE_ADDR 0x4804C000
// địa chỉ base (cơ sở) của thanh ghi điều khiển GPIO1 trên BBB
#define GPIO_SIZE      0x1000
// kích thước vùng thanh ghi (mapping size) là 0x1000 byte (4kB)
#define GPIO_OE        0x134
// OE = Output Enable Register, GPIO_OE = GPIO1_BASE_ADDR + 0x134
#define GPIO_DATAIN     0x138
// dùng để đọc logic từ GPIO, offset đúng.
#define GPIO_DATAOUT    0x13C
// Offset và chức năng ghi logic HIGH/LOW là chính xác.

```

Driver Buzzer (Điều khiển còi cảnh báo):

Driver buzzer được xây dựng nhằm điều khiển bật/tắt còi báo động từ user space thông qua kernel module. Còi báo động được kết nối với một chân GPIO của BeagleBone Black, trong đó việc bật hoặc tắt còi được thực hiện bằng cách ghi mức logic cao hoặc thấp trực tiếp lên thanh ghi DATAOUT.

Trong cấu trúc driver, một node thiết bị ký tự /dev/buzzer được tạo ra để các chương trình ở user space có thể giao tiếp với kernel module. Driver sử dụng hàm write để nhận lệnh điều khiển dưới dạng chuỗi "ON" hoặc "OFF"; khi nhận "ON", driver sẽ kích hoạt còi báo động bằng cách xuất mức logic phù hợp lên GPIO, còn khi nhận "OFF", còi sẽ được tắt.

Ngoài ra, hàm read có thể được sử dụng để trả về trạng thái hiện tại của còi dưới dạng chuỗi "ON\n" hoặc "OFF\n", giúp người dùng kiểm tra trạng thái hoạt động của thiết bị.

Ioremap (ánh xạ GPIO):

```

#define GPIO1_BASE 0x4804C000           // GPIO1
#define GPIO_SIZE 0x1000                // 4KB
#define BUZZER_PIN 28                   // GPIO1_28 => P9_12
#define BUZZER_MASK (1 << BUZZER_PIN)
#define GPIO_OE      0x134              // GPIO_OE = GPIO1_BASE + 0x134
#define GPIO_SETDATAOUT 0x194
// Offset đến thanh ghi SETDATAOUT – ghi 1 vào bit tương ứng để đặt mức HIGH
// (3.3V) cho chân GPIO output
#define GPIO_CLEARDATAOUT 0x190

```

```
// Offset đến thanh ghi CLEARDATAOUT – ghi 1 vào bit tương ứng để đặt mức LOW (0V) cho chân GPIO output
```

Driver MQ2 (Đọc nồng độ khí gas qua ADC ADS1115)

Driver MQ2 được xây dựng nhằm đọc dữ liệu analog từ cảm biến MQ2 thông qua bộ chuyển đổi ADC ADS1115 trên nền tảng BeagleBone Black. Do MQ2 xuất tín hiệu analog nên ADS1115 được sử dụng để chuyển đổi sang dữ liệu số. Driver giao tiếp với ADS1115 bằng giao thức I2C mô phỏng bằng phần mềm (bit-banging) thông qua các chân GPIO SDA và SCL.

Trong cấu trúc driver, node thiết bị ký tự /dev/mq2adc được tạo để user space có thể truy cập cảm biến. Hàm write nhận lệnh "ON" để bắt đầu đọc dữ liệu và "OFF" để dừng cảm biến, trong khi hàm read trả về giá trị ADC hiện tại dưới dạng chuỗi như "ADC: 5432\n". Driver đồng thời thực hiện cấu hình các thanh ghi của ADS1115 để lựa chọn kênh đọc và tốc độ lấy mẫu phù hợp.

Ioremap (ánh xạ gpio):

```
#define GPIO1_BASE 0x4804C000 // GPIO1
#define GPIO_SIZE 0x1000 // 4kB
#define GPIO_OE 0x134 // GPIO_OE = GPIO1_BASE + 0x134
#define GPIO_DATAIN 0x138
#define GPIO_DATAOUT 0x13C
// Offset đến thanh ghi DATAIN – dùng để đọc mức logic hiện tại của các chân GPIO
#define GPIO_DATAOUT 0x13C
// Offset đến thanh ghi DATAOUT – dùng để ghi mức logic ra các chân GPIO output
#define GPIO_SETDATAOUT 0x194
// Offset đến thanh ghi SETDATAOUT – ghi 1 vào bit tương ứng để đặt chân GPIO ở mức HIGH (3.3V)
#define GPIO_CLEARDATAOUT 0x190
// Offset đến thanh ghi CLEARDATAOUT – ghi 1 vào bit tương ứng để đặt chân GPIO ở mức LOW (0V)
```

Ứng dụng giám sát hệ thống – `app.c`

– Chức năng tổng quát

Chương trình `app.c` là ứng dụng giám sát hệ thống môi trường, được viết bằng ngôn ngữ C, chạy ở không gian người dùng (user space) trên nền tảng BeagleBone Black. Chức năng chính của ứng dụng này là đọc dữ liệu từ các cảm biến thông qua driver kernel đã cài sẵn, xử lý dữ liệu, kích hoạt còi cảnh báo nếu phát hiện nguy hiểm, ghi log hệ thống, và duy trì watchdog để đảm bảo hệ thống hoạt động liên tục.

– Chức năng cụ thể

1. Đọc dữ liệu từ cảm biến khí gas MQ2

Ứng dụng giao tiếp với cảm biến MQ2 thông qua bộ ADC ADS1115, được đóng gói dưới dạng driver kernel `/dev/mq2adc`. Chương trình sử dụng hàm `read_mq2()` để mở thiết bị, đọc dữ liệu ADC từ buffer của driver và chuyển đổi thành giá trị số nguyên biểu thị nồng độ khí gas. Dữ liệu nhận được từ driver có định dạng chuỗi "ADC: xxxx" trước khi được xử lý và sử dụng trong hệ thống giám sát.

2. Đọc nhiệt độ và độ ẩm từ cảm biến DHT11

Driver `/dev/dht11` cung cấp dữ liệu nhiệt độ và độ ẩm dưới dạng chuỗi gồm hai dòng thông tin. Ứng dụng sử dụng hàm `read_dht11(&temp, &hum)` để đọc và trích xuất các giá trị này từ buffer của driver. Dữ liệu sau đó được phân tích bằng hàm `sscanf()` nhằm lấy ra giá trị nhiệt độ theo đơn vị °C và độ ẩm theo đơn vị %, phục vụ cho việc giám sát và hiển thị trong hệ thống.

3. Bật/tắt còi báo động thông qua driver buzzer

Hệ thống sử dụng hàm `control_buzzer(int on)` để điều khiển còi báo động thông qua thiết bị `/dev/buzzer` bằng cách ghi chuỗi "ON" hoặc "OFF" vào driver kernel. Buzzer sẽ được kích hoạt khi giá trị nồng độ khí gas đo được vượt quá ngưỡng cảnh báo đã thiết lập trước (`GAS_THRESHOLD = 7000`), giúp hệ thống phát tín hiệu cảnh báo rò rỉ khí gas kịp thời.

4. Hiển thị giá trị cảm biến và trạng thái lên màn hình LCD

Driver LCD2004 được xây dựng nhằm hiển thị thông tin hệ thống lên màn hình LCD thông qua giao tiếp I2C trên nền tảng BeagleBone Black. Driver sử dụng IC mở rộng PCF8574 để điều khiển LCD2004 qua bus I2C và tạo node thiết bị `/dev/lcd2004` để user space có thể gửi dữ liệu hiển thị. Hàm `write` cho phép ghi chuỗi văn bản từ user space xuống LCD, hỗ trợ hiển thị tối đa 4 dòng, đồng thời hỗ trợ lệnh "CLEAR" để xóa màn hình. Driver thực hiện khởi tạo LCD, điều khiển vị trí con trỏ và gửi dữ liệu hiển thị thông qua các lệnh I2C mức thấp.

5. Tích hợp watchdog

Hàm `init_watchdog()` thực hiện mở thiết bị `/dev/watchdog` và cấu hình thời gian timeout là 60 giây. Trong quá trình hoạt động, chương trình định kỳ gọi hàm `keep_alive_watchdog()` để làm mới bộ đếm watchdog, giúp hệ thống không tự động khởi động lại.

Nếu ứng dụng gặp sự cố và không gửi tín hiệu làm mới đúng thời gian quy định, watchdog sẽ tự động reset hệ thống để khôi phục hoạt động. Khi chương trình kết thúc, watchdog được vô hiệu hóa bằng cách ghi ký tự 'V' vào thiết bị `/dev/watchdog`.

6. Ghi log hệ thống

Có hai dạng log:

- Log văn bản: sử dụng **`log_text ()`** ghi vào file `/var/log/sensor_monitor.log` với timestamp kèm theo.
- Log JSON: sử dụng thư viện **`cJSON`**, hàm **`log_json ()`** tạo một chuỗi JSON gồm:
`{"type": "gas", "value": 5432, "timestamp": 1717590000}`

Cả 3 loại dữ liệu (gas, nhiệt độ, độ ẩm) đều được log dạng JSON mỗi chu kỳ.

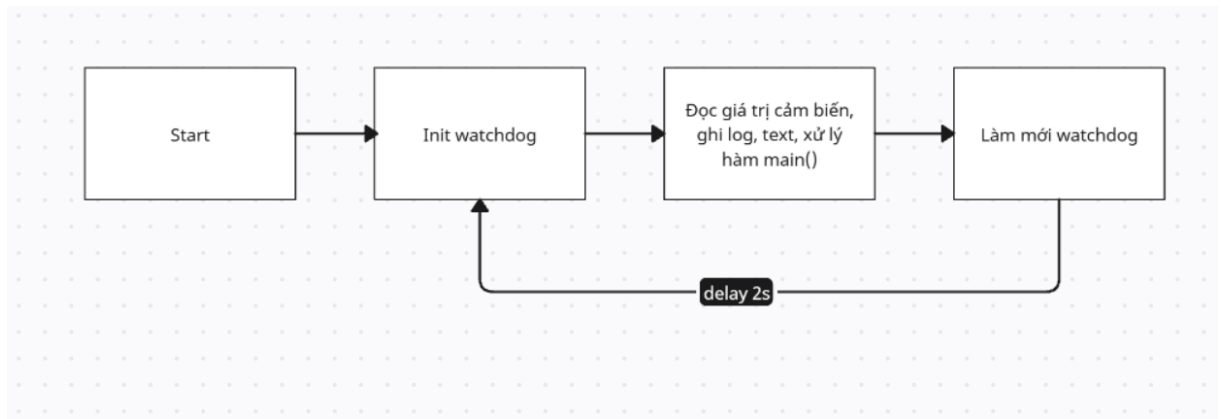
7. Vòng lặp chính

Trong `main()`, chương trình thực hiện các bước:

1. Khởi tạo watchdog.
2. Ghi log khởi động hệ thống.
3. Vào vòng lặp vô hạn:
 - Đọc giá trị khí gas, nhiệt độ, độ ẩm.
 - Ghi log thông tin (text + JSON).
 - Kiểm tra nếu nồng độ khí $\geq$ ngưỡng cho phép. Nếu còi đang tắt $\rightarrow$ bật còi $\rightarrow$ log bật còi.
 - Ngược lại nếu khí gas giảm. Nếu còi đang bật $\rightarrow$ tắt còi $\rightarrow$ log tắt còi.
 - Làm mới watchdog, `sleep(2)` để lặp lại sau 2 giây.

4. Khi kết thúc (dù logic hiện tại là vô hạn), hệ thống sẽ gọi **`disable_watchdog()`** để dừng watchdog.

Luồng hoạt động chính



2.2.5. Autostart và đóng gói buildroot

Autostart init.d

Sau khi xây dựng thành công application và các driver trên hệ thống Buildroot, cơ chế init.d được sử dụng để tự động khởi động application và nạp các driver ngay khi hệ thống boot. Một file script có tên S99appsensor được tạo trong thư mục /etc/init.d/, trong đó tiền tố S99 biểu thị thứ tự khởi động của dịch vụ trong hệ thống Buildroot, với giá trị có thể từ S00 đến S99. Script này có nhiệm vụ nạp các kernel module cần thiết và tự động chạy chương trình giám sát cảm biến khi BeagleBone Black khởi động.

Đóng gói Buildroot

Quá trình đóng gói hệ thống bằng Buildroot được thực hiện tự động sau khi toàn bộ driver và application đã được thêm vào thư mục package. Người dùng chỉ cần truy cập terminal của Buildroot và thực thi lệnh make, khi đó hệ thống sẽ tự động biên dịch source code, build kernel module, application và tạo ra bộ file hoàn chỉnh trong thư mục /output/images.

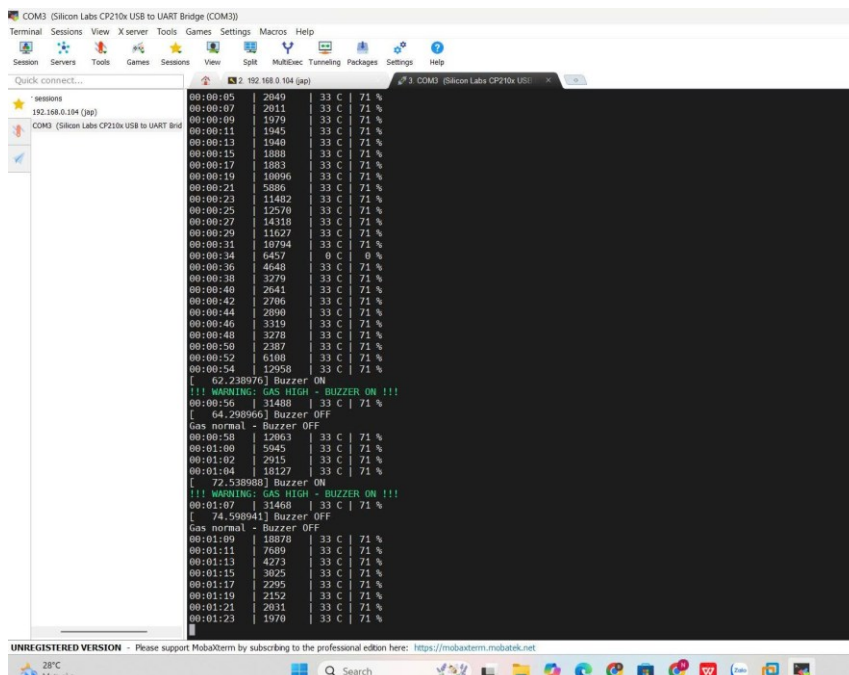
Thư mục này chứa các file cần thiết để boot hệ thống cùng với file rootfs.tar, trong đó đã bao gồm toàn bộ driver, application và các thành phần được tích hợp trong Buildroot, cho phép dễ dàng triển khai sang thiết bị khác. Ngoài ra, khi cần chỉnh sửa driver hoặc application, chỉ cần cập nhật lại file source .c rồi thực hiện make lại để tạo ra phiên bản mới có thể sử dụng ngay trên hệ thống.

CHƯƠNG 3: XÂY DỰNG HỆ THỐNG

3.1. Chức năng chính của hệ thống

Hệ thống giám sát và cảnh báo rò rỉ khí gas sử dụng BeagleBone Black được thiết kế nhằm đảm bảo khả năng hoạt động ổn định, giám sát thời gian thực và tự động cảnh báo khi phát hiện nồng độ khí gas vượt ngưỡng. Các chức năng chính của hệ thống bao gồm:

- Thu thập dữ liệu từ cảm biến khí gas MQ2: Đọc giá trị nồng độ khí gas (ở dạng ADC) thông qua mô-đun ADC (như ADS1115) và xử lý giá trị này trong ứng dụng người dùng.
- Đọc nhiệt độ và độ ẩm môi trường từ cảm biến DHT11: Kết hợp thông tin nhiệt độ và độ ẩm để cung cấp bức tranh tổng quan về điều kiện môi trường.
- Cảnh báo nguy hiểm bằng còi (buzzer) và hiển thị lên LCD: Khi nồng độ khí gas đo được vượt quá ngưỡng an toàn định sẵn, hệ thống sẽ kích hoạt còi cảnh báo đồng thời hiển thị giá trị cảm biến hiện tại và trạng thái báo hiệu.
- Xử lý và ghi log dữ liệu: Ứng dụng ghi lại dữ liệu đọc được và trạng thái cảnh báo vào tệp nhật ký trong hệ thống file để phục vụ cho việc giám sát và phân tích sau này.
- Tự động khởi động sau khi cấp nguồn: Hệ thống sử dụng cơ chế init.d để tự động tải driver kernel và khởi chạy ứng dụng appsensor ngay khi bật nguồn thiết bị.



The screenshot shows a terminal window with a title bar indicating it's connected to a COM3 (Silicon Labs CP210x USB to UART Bridge). The terminal displays a continuous stream of data in a tabular format, including timestamps, numerical values, and units. Key events include 'Buzzer ON' and 'WARNING: GAS HIGH - BUZZER ON !!!' at 00:00:54 and 00:01:07, followed by 'Buzzer OFF' and 'Gas normal - Buzzer OFF' at 00:00:56 and 00:01:09 respectively. The bottom of the window shows a Windows taskbar with a search bar and various application icons.

```
COM3 (Silicon Labs CP210x USB to UART Bridge (COM3))
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split Multitab Tunneling Packages Settings Help
Quick connect...
sessions
192.168.0.104 (jag)
COM3 (Silicon Labs CP210x USB to UART Bridge)

00:00:05 | 2040 | 33 C | 71 %
00:00:07 | 2011 | 33 C | 71 %
00:00:09 | 1979 | 33 C | 71 %
00:00:11 | 1945 | 33 C | 71 %
00:00:13 | 1940 | 33 C | 71 %
00:00:15 | 1888 | 33 C | 71 %
00:00:17 | 1883 | 33 C | 71 %
00:00:19 | 1896 | 33 C | 71 %
00:00:21 | 5886 | 33 C | 71 %
00:00:23 | 11482 | 33 C | 71 %
00:00:25 | 12570 | 33 C | 71 %
00:00:27 | 14318 | 33 C | 71 %
00:00:29 | 11627 | 33 C | 71 %
00:00:31 | 10794 | 33 C | 71 %
00:00:34 | 6457 | 0 C | 0 %
00:00:36 | 4648 | 33 C | 71 %
00:00:38 | 3279 | 33 C | 71 %
00:00:40 | 2641 | 33 C | 71 %
00:00:42 | 2706 | 33 C | 71 %
00:00:44 | 2890 | 33 C | 71 %
00:00:46 | 3319 | 33 C | 71 %
00:00:48 | 3270 | 33 C | 71 %
00:00:50 | 2387 | 33 C | 71 %
00:00:52 | 6100 | 33 C | 71 %
00:00:54 | 12958 | 33 C | 71 %
[ 62.238976] Buzzer ON
!!! WARNING: GAS HIGH - BUZZER ON !!!
00:00:56 | 31480 | 33 C | 71 %
[ 64.208966] Buzzer OFF
Gas normal - Buzzer OFF
00:00:58 | 12063 | 33 C | 71 %
00:01:00 | 5045 | 33 C | 71 %
00:01:02 | 2915 | 33 C | 71 %
00:01:04 | 18127 | 33 C | 71 %
[ 72.538988] Buzzer ON
!!! WARNING: GAS HIGH - BUZZER ON !!!
00:01:07 | 31468 | 33 C | 71 %
[ 74.508941] Buzzer OFF
Gas normal - Buzzer OFF
00:01:09 | 18878 | 33 C | 71 %
00:01:11 | 7689 | 33 C | 71 %
00:01:13 | 4273 | 33 C | 71 %
00:01:15 | 3025 | 33 C | 71 %
00:01:17 | 2295 | 33 C | 71 %
00:01:19 | 2152 | 33 C | 71 %
00:01:21 | 2031 | 33 C | 71 %
00:01:23 | 1970 | 33 C | 71 %

UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
28°C
Mùa rào
```


Hình 3.1. Giá trị cảm biến đọc được qua UART



Hình 3.2. Giao diện đóng gói hệ thống

3.2. Thử nghiệm thực tế hệ thống

Sau khi biên dịch và triển khai hoàn chỉnh hệ thống lên BeagleBone Black, nhóm đã tiến hành thử nghiệm thực tế trong môi trường nhà ở và thu được kết quả khả quan.

Hệ thống hoạt động ổn định trong thời gian dài mà không xảy ra hiện tượng treo máy hay lỗi đọc cảm biến. Các cảm biến MQ2 và DHT11 cho dữ liệu đầu ra khá chính xác, ứng dụng xử lý và cập nhật dữ liệu nhanh chóng theo thời gian thực. Buzzer phản hồi đúng với điều kiện cảnh báo đã cấu hình, ví dụ khi nồng độ khí gas vượt quá ngưỡng $PPM > 5200$ thì còi sẽ tự động kích hoạt.

Ngoài ra, hệ thống còn hỗ trợ ghi log đầy đủ thông tin như thời gian, giá trị cảm biến và trạng thái cảnh báo để phục vụ việc giám sát. Cơ chế tự động khởi động thông qua init.d hoạt động hiệu quả, giúp hệ thống tự chạy ngay sau khi cấp nguồn mà không cần thao tác thủ công.

CHƯƠNG 4: KẾT QUẢ, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Trong khuôn khổ đề tài, nhóm đã xây dựng thành công hệ thống giám sát cảm biến trên nền tảng BeagleBone Black sử dụng hệ điều hành Linux tùy chỉnh được tạo bằng Buildroot. Hệ thống bao gồm các driver nhân Linux cho cảm biến MQ2 thông qua ADC ADS1115, cảm biến DHT11, LCD 20x04 và buzzer cảnh báo. Đồng thời, một ứng dụng ở user space được phát triển để đọc dữ liệu cảm biến, xử lý thông tin và điều khiển thiết bị theo logic an toàn đã thiết lập.

Việc giao tiếp giữa user space và kernel space được thực hiện thông qua các node thiết bị /dev với các lệnh điều khiển đơn giản như "ON", "OFF" hoặc cat. Qua quá trình thử nghiệm, hệ thống cho thấy khả năng hoạt động ổn định, dễ bảo trì và có thể mở rộng linh hoạt theo yêu cầu thực tế. Bên cạnh đó, việc triển khai driver ở mức kernel giúp tối ưu hiệu suất xử lý, giảm độ trễ phản hồi và nâng cao hiệu quả hoạt động trong môi trường nhúng có tài nguyên hạn chế.

4.2. Hướng phát triển

Để mở rộng và nâng cao tính ứng dụng của đề tài, hệ thống có thể được phát triển thêm theo nhiều hướng khác nhau. Trước hết, có thể tích hợp các giao thức mạng như MQTT hoặc TCP/IP để gửi dữ liệu cảm biến lên máy chủ giám sát, hệ thống SCADA hoặc nhận lệnh điều khiển từ xa thông qua web.

Bên cạnh đó, việc xây dựng giao diện người dùng HMI dưới dạng web hoặc ứng dụng di động sẽ giúp người dùng theo dõi trạng thái hệ thống theo thời gian thực một cách trực quan hơn. Hệ thống cũng có thể bổ sung cơ chế cảnh báo tự động qua email, SMS hoặc thông báo đẩy khi giá trị cảm biến vượt ngưỡng nguy hiểm, đồng thời lưu trữ log dữ liệu để phục vụ phân tích sau này. Ngoài các cảm biến hiện tại, đề tài có thể mở rộng thêm các loại cảm biến khác như MQ7, DS18B20, BH1750,... kết hợp với việc phát triển driver tương ứng.

Cuối cùng, ở mức hệ điều hành, có thể tối ưu kernel bằng cách loại bỏ các thành phần không cần thiết, rút ngắn thời gian khởi động và tích hợp watchdog nhằm tăng độ tin cậy khi hệ thống vận hành liên tục trong thời gian dài.

PHỤ LỤC: MÃ NGUỒN & GITHUB

- **Link Code:**

https://github.com/nguyencanh178/Embedded_OS/tree/main/BTL



TÀI LIỆU THAM KHẢO

- [1] Buildroot Developers, "Buildroot User Manual – Custom Packages", *buildroot.org*, [Online]. Available: <https://buildroot.org/downloads/manual/manual.html#custom-packages>
- [2] BusyBox Team, "BusyBox Init System", *busybox.net*, [Online]. Available: <https://busybox.net/downloads/BusyBox.html>
- [3] P. Jayaraman, "Linux Kernel Module Programming Guide", *The Linux Documentation Project*, [Online]. Available: <https://tldp.org/LDP/lkmpg/2.6/html/>
- [4] eLinux Wiki, "BeagleBone Black Development", *elinux.org*, [Online]. Available: https://elinux.org/Beagleboard:BeagleBone_Black_Development
- [5] G. Beekmans et al., "Linux From Scratch – Boot Scripts", *linuxfromscratch.org*, [Online]. Available: <https://www.linuxfromscratch.org/lfs/view/development/chapter07/bootscripts.html>

